Step 1: Frame the visitor service problem

Purpose: Turn the sponsor request into an evidence-linked business problem and a proportionate first slice.

LEARNER LAB TASK

1. Open the online VS Code workspace from this platform. Confirm that the evidence and work folders are visible. Read evidence/cinema-brief.md.
2. In work/ai-ready-specification.md, complete Business problem and desired outcome. Distinguish a supplied observation, an assumption and a missing fact. Describe potential business value in plain language without inventing savings or revenue.
3. Complete Scope of the first slice. Choose a bounded staff-facing use of AI assistance. State what remains outside it, including decisions about bookings, refunds and sending messages. Record the same first judgement in work/review-record.md.
4. Respond to the lab question in the platform poll or meeting chat: Which first slice would you take forward: information drafts for staff review, automatic booking changes, or autonomous refund decisions? Give one reason linked to the brief. If a poll is unavailable, use the meeting chat; if you cannot post, record your choice in the review record and use the platform help route.

Checkpoint: Your problem describes the visitor service difficulty rather than “we need AI”. Your proposed slice has a named user, a useful output and a clear boundary.

If blocked: If the workspace is empty or files are absent, use the platform help route and report the missing folder. Do not recreate evidence from memory. While access is restored, write the problem and scope in the platform response area; transfer them into the workspace when it is available.

Expected output: An evidence-linked problem, desired outcome, assumptions and a bounded first slice.

Assessment check: The problem and value follow from supplied evidence; the scope does not authorise unapproved customer actions.

Reflection: Which part of the sponsor request is a business need, and which part is a proposed solution?

Step 2: Decompose the service into tasks

Purpose: Choose AI assistance at task level while retaining rules and human judgement where appropriate.

LEARNER LAB TASK

1. Read the source records in evidence/approved-visitor-information.csv and the messages in evidence/visitor-queries.csv. CSV files can be read and edited as text in the workspace; no spreadsheet application is required.
2. Replace the six blank rows in work/task-map.csv with the work needed from receiving a visitor question to staff approval. You may add, combine or remove rows if the responsibilities remain clear.
3. For each task, specify its purpose, required input, expected output, approach and reason. Distinguish ordinary rules, AI assistance and human judgement. Include evidence selection, exception handling and final review rather than making every task an AI task.
4. Identify the handoff or stop condition for every task. In Behaviour and responsibilities in your specification, summarise the resulting sequence and name the role that owns the final decision.

Checkpoint: Another person can follow the work from question to reviewed draft. Every task has an input and output, and your rationale explains why an AI component would help with the chosen slice.

If blocked: Use the brief to identify verbs such as receive, select, interpret, draft, check and approve. Keep only tasks that produce a needed output. If CSV quoting becomes difficult, enclose any cell containing a comma in double quotation marks or use a simple Markdown table with the same column headings and save it as work/task-map.md; note the change in the review record.

Expected output: A justified task map with explicit rule, AI-assisted and human responsibilities.

Assessment check: Tasks are specific enough to build or allocate; each has an input, output, rationale and failure route.

Reflection: Which task would become less reliable or less accountable if it were assigned to AI without a boundary?

Step 3: Define the input and output contract

Purpose: Make the bounded drafting task precise enough for a business and engineering handoff.

LEARNER LAB TASK

1. Complete Inputs and evidence selection in work/ai-ready-specification.md. Define the required fields and their meanings. Include the visitor question, relevant venue or screen, and permitted evidence. State what happens if required context is missing.
2. State how approved records are selected and archived records excluded. Define the rule for conflicting approved facts. Do not silently choose whichever fact produces the most convenient answer.
3. Complete Output contract. Define a small set of statuses, a draft field, source references and a staff handoff field. Use your own descriptive names if you prefer. State which fields must be present or empty for each status.
4. Complete the behaviour boundaries. State exactly what may be drafted, what must never be claimed or done, who reviews the draft, and how requests involving private details or unsupported facts are routed.
5. Write observable criteria in Success and acceptance. Replace words such as “accurate”, “safe” or “helpful” with a visible check. For example, specify that every factual claim in an eligible draft must be supported by a named approved source record.

Checkpoint: A colleague can determine the required output for a supported question, a missing venue, a source conflict and a booking action without asking you to explain an unwritten rule.

If blocked: Keep the contract small. A staff-facing outcome may be a grounded draft, a clarification request or a staff review handoff. An exception does not need a customer-ready draft. Record an unresolved business decision with its owner rather than inventing a policy.

Expected output: Explicit input fields, output fields, statuses, evidence rules, human boundaries and observable success criteria.

Assessment check: The contract defines normal and exception outcomes and forbids unsupported claims or unauthorised actions.

Reflection: What would an engineer still have to guess from your current wording?

Step 4: Write and walk through acceptance cases

Purpose: Expose ambiguity with concrete normal and exception inputs before implementation.

LEARNER LAB TASK

1. In work/acceptance-cases.csv, fully specify at least six of the supplied requests. Include a supported information question, a booking action, missing context, conflicting access information, private details and an unsupported fact. The instruction-to-ignore-sources case is an additional useful challenge.
2. For each case, define the exact input, expected status, required output or behaviour, source evidence and an observable pass condition. Avoid criteria such as “the answer looks good”.
3. Walk each selected input through your written specification. This is a manual specification walkthrough, not a model execution. Record the predicted route and any point where the contract leaves the outcome ambiguous in Walkthrough observation. Record the requirement to change in Revision needed.
4. Check at least one apparently helpful answer that must be held back. The conflicting Screen 3 records and unsupported screening date are useful candidates. Refer to both sides of a conflict in the evidence column.

Checkpoint: At least six rows can be used by another person to judge a later implementation. Each row states what must be present, absent or routed, and its pass condition can be observed.

If blocked: If there is no fact for the requested date, do not invent one. If two approved records disagree, record the conflict rather than treating approval as proof of agreement. If CSV editing causes difficulty, use work/acceptance-cases.md with the same headings and note the alternative in the review record. Preserve your earlier reasoning when revising a case.

Expected output: At least six concrete acceptance cases with explicit predicted outcomes and recorded specification gaps.

Assessment check: Normal, action, missing-context, conflicting-evidence, private-data and unsupported-fact paths are distinguishable and testable.

Reflection: Which case revealed the largest gap between an intention and a buildable requirement?

Step 5: Challenge the draft and revise the specification

Purpose: Use evidence and a meaningful trade-off to improve the specification rather than accepting fluent output.

LEARNER LAB TASK

1. Read evidence/supplied-draft-fixtures.md. The outputs are supplied fictional fixtures; no model was run. Critique each against the evidence and your contract in work/review-record.md.
2. Identify an unsupported claim, an unauthorised action and a missing workflow field. Explain the specific requirement that would expose or prevent each issue. Do not treat polished wording as evidence that a claim is supported.
3. Consider the trade-off between answering more queries and keeping the first slice controllable. Post one sentence in the meeting chat: I would include ___ and hold back ___ because the evidence shows ___. Read one other contribution and record whether it changes your decision. If chat is unavailable, record the sentence and your self-challenge in the review record and use the platform help route.
4. Make at least one substantive revision to your task map, contract or acceptance cases. In the review record, preserve the earlier wording or describe it accurately, identify what exposed the gap and explain the improvement. If your first version already covers every supplied fixture, add a new boundary case and tighten a criterion that it exposes.
5. Update related acceptance cases so the revised requirement has an observable check. Record any remaining decisions and a responsible role in the specification.

Checkpoint: The review record shows a concrete reasoned change, and the revised requirement is reflected in an acceptance case.

If blocked: Use the archived access advert to challenge evidence selection, the booking promise to challenge action authority, or the incomplete draft to challenge required output fields. Keep uncertainty visible; do not resolve a fictional source conflict by searching for a real cinema policy.

Expected output: Critique of supplied fixtures, a documented scope trade-off and a justified specification revision.

Assessment check: The revision changes a buildable rule, contract, responsibility or criterion and is supported by a corresponding case.

Reflection: What useful capability did you defer, and what evidence would justify adding it later?

Step 6: Review and submit the engineering handoff

Purpose: Produce a coherent handoff that a team can review before implementing a prototype.

LEARNER LAB TASK

1. Use the completion checks in work/review-record.md. Check that the problem, scope, task map, contract and acceptance cases describe the same service. Correct any contradictory status name, responsibility or output requirement.
2. Confirm that at least six acceptance cases are complete and include the required normal and exception paths. Confirm that your review record explains a trade-off, a critique and a substantive revision.
3. State the decisions still needed from the business or engineering team and the role responsible for each. Keep later prototype evaluation separate from this specification walkthrough. There is no live model result to report.
4. Save the four files in work. Record your name or platform identifier and the submitted filenames in the review record. Submit the four files through the platform's course submission control. If that control is unavailable, use the platform help route and provide the saved workspace paths; keep the workspace open and retain the files until collection is confirmed. Do not post the full work into public chat.

Completion standard: Your handoff links the business problem to a justified task decomposition, defines actionable inputs and outputs, states human boundaries, includes at least six observable acceptance cases, and records an evidence-based revision. A different design is acceptable if it meets these criteria and explains its choices.

Reflection: Which business problem from your own work could use the same sequence of problem, task, contract and evidence checks? This reflection remains part of the lab; the lecturer-led close follows submission.

Recovery and persistence: If you lose connection, re-enter the same platform workspace and check your saved files before editing again. Persistence and submission behaviour depend on the deployed platform and must be validated by its operator. If a file was damaged, preserve it under a new descriptive filename before using the platform reset route. A reset may restore the starter and lose unsaved work, so ask platform support before resetting. If support restores a fresh workspace, transfer your preserved work and repeat the affected checks; do not mark an unreviewed file complete.

Expected output: Four saved and submitted work products with a coherent specification and review trail.

Assessment check: All five rubric criteria are met; remaining prototype and business decisions are explicitly assigned.

Reflection: Which requirement would you ask a stakeholder to confirm first before development?